

Forecasting Six Weeks of Rossmann Sales: An End-to-End Case Study

A NextHikes IT Solutions project for Rossmann Pharmaceuticals

The problem

Rossmann Pharmaceuticals runs 1,115 drug stores across Germany, and until now, forecasting next month's sales has meant asking each store manager to make an educated guess. That doesn't scale, it isn't auditable, and it can't easily account for the interacting effects of promotions, competition, holidays and seasonality that everyone agrees matter but nobody had actually quantified.

This project builds the alternative: an end-to-end pipeline that cleans and understands 2.5 years of daily sales history, engineers the features that actually predict demand, compares a tree-based model against a gradient-boosted one and a deep-learning forecaster, and serves the result through a dashboard the finance team can use directly -- no notebook, no data science degree required.

The data

Three files from the original Kaggle Rossmann Store Sales competition -- train.csv, test.csv, store.csv -- plus three locality extras (state, daily weather, weekly Google search-trend index) pulled in to give the "locality" factor the brief called out its own features. In total: 1,017,209 daily records across 1,115 stores, January 2013 to July 2015, with the test window running exactly the brief's 48-day forecast horizon.

Two data traps shaped the whole pipeline. First, Customers -- the single strongest predictor of Sales ($r = 0.82$) -- does not exist in the test set, because footfall isn't known before the day happens; using it would produce a great validation score and a model that can never be deployed. Second, Open == 0 days always have Sales == 0: that's a business rule, not a pattern to learn, so closed days are excluded from training and the rule is imposed directly at inference time.

What the data actually says (Task 1)

Fourteen questions were asked of the data -- eleven from the brief, three of our own -- each answered with a chart, an evidence table and a written conclusion (reports/eda_findings.md has the full set). Three findings changed the shape of the model:

Store identity dominates. Decomposing total sales variance, 60% of it sits between stores rather than within a store's own day-to-day fluctuation. Store size varies sixfold across the estate. That single fact drove two decisions: per-store historical aggregates (mean/median sales, weekday and month baselines) became the highest-value feature family, and RMSPE -- a relative, scale-invariant error metric -- was chosen over RMSE specifically because a scale-sensitive loss would let the largest stores dominate the objective and effectively ignore the smallest ones.

Promos work through both channels, roughly equally. Restricting to weekdays (promos never run on weekends), promo days deliver a 38.8% sales lift, splitting into a 19.8% footfall increase and a 15.3% basket-size increase -- genuinely more customers and bigger baskets, not one masquerading as the other. But the response is wildly uneven: the top quartile of stores captures 38% of all incremental promo revenue, the bottom quartile just 14%. That's an immediately actionable recommendation independent of any model: retarget promo spend toward the stores that actually respond to it.

Competitor distance measures urbanity, not competition. Taken at face value, stores with a competitor inside 250m sell more than stores with no nearby competitor at all -- backwards from the obvious hypothesis. The resolution: competitors cluster where customers are, so distance is mostly a proxy for how central a store's location is. Splitting on a 500m city-centre proxy and re-testing the correlation within each locality type collapses it to near zero, confirming the raw distance value was measuring density, not competitive pressure. It's kept as a log-transformed feature alongside an explicit city-centre flag so the model can separate the two readings.

One correction is worth stating plainly, because it's a useful lesson in checking assumptions against the data rather than against intuition: the initial pass assumed the December sales peak sat on 22-24 December, the obvious "Christmas Eve rush" story. The actual day-by-day figures say otherwise -- the single highest-selling day of the year lands in the third week of Advent, and Christmas Eve itself is a below-average half-trading day, almost certainly a shortened trading day before the holiday closure. The lesson generalises: day-level features (not month, not a coarse "is-December" flag) are what the model needs to represent this correctly.

Building the model (Task 2)

Preprocessing and features live in one place (`src/features.py`) and run inside a single `sklearn.pipeline.Pipeline` (`src/pipeline.py`) -- cleaning, feature engineering, scaling and encoding are one fittable, picklable object, so the exact code that trains the model is the code that serves it in production. Beyond the calendar features the brief asks for (weekday/weekend, days to/from a holiday, month phase), the pipeline adds cyclical sin/cos encodings, a decoded `IsPromo2Active` flag (parsed from strings like "Feb,May,Aug,Nov"), months-since-competitor-opened, and the per-store historical aggregates that Task 1 identified as the dominant signal.

Validation is the final six weeks of the training window, not a random split -- matching the brief's actual forecast horizon and avoiding the leakage that would come from a store's own future statistics bleeding into its past.

The loss function is RMSPE (Root Mean Square Percentage Error), chosen and defended in `src/metrics.py`: it treats a 10% miss on a corner store as seriously as a 10% miss on the flagship, which a currency-denominated metric like RMSE would not, and it's directly interpretable to a non-technical stakeholder ("forecasts are within N% on average").

Two models were compared behind the identical pipeline -- a `RandomForestRegressor` (the brief's suggested starting point) and a `LightGBM` gradient booster (the "innovative approach" upgrade), both trained on $\log_{1p}(\text{Sales})$ and validated on the held-out final six weeks of the training window (804,056 train rows / 40,282 validation rows):

Model	RMSPE	RMSE	MAE	MAPE	R ²	Fit time	Serialized size
Random Forest (150 trees, depth 14)	0.1395	1,060.7	721.8	10.41%	0.879	180s	89.7 MB
LightGBM (600 trees, 63 leaves)	0.1294	929.7	635.8	9.45%	0.907	106s	4.3 MB

LightGBM wins outright -- lower error on every metric, faster to train, and roughly 20x smaller on disk -- and is the model the Flask app serves by default as a result. The Random Forest earns its place anyway: its top permutation-importance features (Store, Promo, Date, then CompetitionDistance/Promo2SinceYear) confirmed the same store-identity and calendar dominance Task 1 identified independently, and it's kept loadable in the app specifically for the confidence-interval capability below, which needs an explicit tree ensemble.

One engineering note that shaped the final hyperparameters directly: the first Random Forest configuration (min_samples_leaf=2, no depth cap) trained to a marginally better RMSPE (0.1314) but serialized to a 3.08 GB pickle file -- completely unfit for git, a deployment target, or fast cold-start loading. Capping min_samples_leaf=25 and max_depth=14 cost 0.008 of RMSPE and bought a 34x smaller, 3.4x faster-to-train model. On a forecasting product that has to actually ship, that trade is not a close call.

Post-prediction analysis used permutation importance rather than the Random Forest's built-in feature_importances_, which is biased toward high-cardinality columns, and produced confidence intervals essentially for free: a Random Forest already trains 150 independent trees, so the spread of their individual predictions for a row is a genuine, zero-extra-cost estimate of the model's own uncertainty. Measured empirical coverage of the nominal 95% interval was 78.1% on the validation set -- under-covering, as expected, since tree-to-tree disagreement captures only the model's own uncertainty and misses irreducible noise in the data. That's reported here rather than smoothed over: the interval is genuinely useful for ranking which forecasts to trust least, but should not be read as a calibrated 95% bound without a wider correction factor.

Models are serialized with a timestamp (rf_DD-MM-YYYY-HH-MM-SS-CC.pkl), on the assumption of daily retraining -- every version is independently addressable and traceable back to the predictions it produced.

Deep learning (Task 2.6)

A two-layer LSTM was built following the brief's full seven-step process: isolate a time series, test for stationarity (Augmented Dickey-Fuller), difference if the test calls for it, inspect ACF/PACF to choose a lookback window, transform the series into supervised sliding-window examples, scale to (-1, 1), and train the network.

The national daily-sales series tested stationary by the Augmented Dickey-Fuller test (statistic = -4.76,

$p = 6.4 \times 10^{-5}$), so no differencing was applied -- differencing is offered in `src/lstm_model.py` and would trigger automatically had the test said otherwise. ACF and PACF both show sharp, clean spikes at lags 7, 14 and 21 and near-zero everywhere else -- the weekly trading cycle dominates the series' structure so completely that it's visible without any modelling at all. That directly motivated a 14-day lookback window: two full cycles of context, enough for the network to compare "this Monday" against both of the last two Mondays.

Rather than fit on that single aggregate series -- which would give only ~900 training points, too few to trust a neural network's fit -- the sliding windows were built per store (120 stores sampled) and stacked into 83,954 training sequences, each store scaled independently with its own `MinMaxScaler` (store size varies sixfold, so a single global scale would let the largest stores dominate the loss exactly as RMSE would).

The resulting two-layer network (LSTM(50) -> LSTM(25) -> Dense(16) -> Dense(1), 18,433 parameters total, 72 KB) trained in 338 seconds on a CPU -- comfortably within the brief's "should run in Colab" constraint -- and reached:

Model	RMSPE	RMSE	MAE	R ²
LSTM (14-day window, 120 stores)	0.1606	1,092.8	766.3	0.886

That's behind both tree models (LightGBM 0.1294, Random Forest 0.1395), and the reason is structural rather than a modelling shortfall: the LSTM here is deliberately univariate -- it only ever sees a store's own past sales, with no Promo, no calendar features, no competitor or Promo2 information, and critically no store-identity signal beyond what's implicit in its own recent history. Task 1 established that store identity alone explains ~60% of total sales variance; a model that can't be told which store it's looking at is giving up access to the single strongest predictor in the dataset before it starts. That the LSTM still reaches $R^2 = 0.886$ from sales history alone is a reasonable result for a sequence-only model, and the natural next iteration -- feeding the same engineered feature set into the network as auxiliary inputs alongside the sales sequence -- is the direct route to closing the gap with the tree models.

Serving it (Task 3)

The trained pipeline is served through a small Flask dashboard (app/): a finance analyst picks a store, uploads a CSV of the dates they want forecast (with optional holiday/promo flags), and gets back a chart of predicted sales -- with a confidence band, where the underlying model supports it -- an estimated customer count, and a CSV they can download and take into their own planning process. No preprocessing code is duplicated between training and serving; the dashboard calls the exact same pipeline object that was validated in Section 2.

What the finance team should do with this

1. Retarget promo spend toward the top-quartile-responding stores identified in Task 1 -- this doesn't require the model at all and can be actioned immediately.

2. Treat competitor-distance data with caution in any manual override of the model's forecasts; it mostly encodes location type, not competitive threat.
3. Adopt the model for the six-week forecast cycle it was built for, with the caveat that the current validation window (August-September) can't directly confirm accuracy during the December peak -- worth a dedicated check once a full year of live forecasts has accumulated.
4. Retrain regularly using the existing pipeline and timestamped serialization -- the infrastructure to do this daily, with full MLflow tracking, is already in place.